PYTHON · INFORMATIK

Einführung ins Programmieren mit Python

Grundlagen der Informatik

Theresa Luternauer · Julia Imhof

HS 2026/27

Kantonsschule Uster

Einführung ins Programmieren mit Python

Was ist ein Algorithmus, und wie führt ein Computer unsere Anweisungen aus? Du lernst die Grundlagen der Programmierung kennen, schreibst erste Python-Anweisungen mit `turtle` und erfährst, wie du Fehler erkennst und behebst.

Lernziele

Nach diesem Kapitel kannst du …

- erklären, was ein Algorithmus, ein Computerprogramm und eine Programmiersprache sind.
- die Begriffe **Syntax** und **Semantik** unterscheiden.
- ein erstes Python-Programm mit dem Modul `turtle` lesen und ausführen.
- Kommentare in Python sinnvoll einsetzen.
- einfache Fehler in Programmen erkennen und systematisch suchen.

Was ist ein Algorithmus?

Ein **Algorithmus** ist eine präzise Anleitung zur Lösung eines Problems oder zur Ausführung einer Aufgabe. Er besteht aus einer Folge von einzelnen Anweisungen, die so eindeutig formuliert sind, dass sie Schritt für Schritt ausgeführt werden können.

Auch im Alltag begegnen uns Algorithmen, zum Beispiel:

- eine Anleitung zum Zusammenbau eines Regals,
- eine Gebrauchsanweisung.

In digitalen Geräten und Anwendungen werden Algorithmen in Programmiersprachen formuliert. Sie kommen zum Beispiel in Computern, Smartphones, Navigationsgeräten, Ampeln, Fahrassistenzsystemen, Suchmaschinen und Online-Shops zum Einsatz.

> **Merke:**
> Ein Algorithmus beschreibt **Schritt für Schritt**, wie ein Problem gelöst oder eine Aufgabe ausgeführt wird.

Was ist ein Computerprogramm?

Ein **Computerprogramm** ist ein Algorithmus, der in einer Programmiersprache formuliert wurde und von einem Computer ausgeführt werden kann.

Programmieren bedeutet also, einem Computer genaue Anweisungen zu geben. Der Computer führt diese Anweisungen in der vorgegebenen Reihenfolge aus.

Was ist eine Programmiersprache?

Eine **Programmiersprache** ist eine Sprache, mit der Programme für einen Computer formuliert werden. Sie besitzt einen festgelegten Wortschatz und genaue Regeln.

Es gibt viele verschiedene Programmiersprachen. Sie unterscheiden sich unter anderem in ihrer Schreibweise und in den Bereichen, für die sie häufig eingesetzt werden.

In diesem Kurs verwenden wir **Python**.

Python

Python wurde Anfang der 1990er-Jahre von Guido van Rossum entwickelt. Der Name bezieht sich auf die britische Comedy-Gruppe *Monty Python*. Heute wird Python von der Python Software Foundation weiterentwickelt und gepflegt.

Python wird in vielen unterschiedlichen Bereichen eingesetzt. Dazu gehören unter anderem Webanwendungen, Datenverarbeitung, Automatisierung und Machine Learning.



Gründer von Python: Guido van Rossum mit dem bekannten Logo von Python

---

Syntax und Semantik

Beim Programmieren muss zwischen **Syntax** und **Semantik** unterschieden werden.

Syntax

Die **Syntax** beschreibt die Regeln, nach denen Befehle in einer Programmiersprache geschrieben werden müssen. Sie entspricht ungefähr der Grammatik einer natürlichen Sprache.

Beispiel:

```
t.forward(50)
```

Dieser Befehl besitzt eine korrekte Python-Syntax.

```
t.forward 50
```

Hier fehlen die Klammern. Python kann den Befehl deshalb nicht korrekt verarbeiten und gibt eine Fehlermeldung aus.

Semantik

Die **Semantik** beschreibt die Bedeutung eines Befehls oder Programms: Was bewirkt der Code tatsächlich?

Beispiel:

```
print("Hallo")
```

Die Syntax ist korrekt. Die Semantik des Befehls ist: Der Text `Hallo` wird ausgegeben.

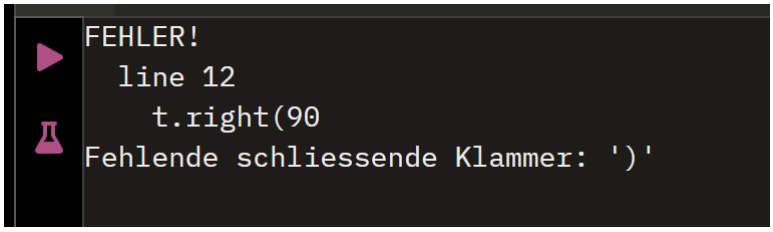
> **Merke:**
> **Syntax** = Wie wird etwas korrekt geschrieben?
> **Semantik** = Was bedeutet der Code und was bewirkt er?

Fehler gehören zum Programmieren

Beim Programmieren entstehen häufig Fehler. Das ist normal. Ein Computer interpretiert Anweisungen jedoch nicht so flexibel wie ein Mensch. Stimmen die Regeln der Programmiersprache nicht, kann ein Programm nicht wie vorgesehen ausgeführt werden.

Enthält ein Programm einen Syntaxfehler, zeigt die Entwicklungsumgebung normalerweise eine **Fehlermeldung** an. In CodeExpert findest du diese Meldungen in der **Konsole**.

Eine **Entwicklungsumgebung** ist ein Programm oder eine Webseite, in der Programmcode geschrieben, ausgeführt und getestet werden kann.



Fehlermeldung bei einem Turtle-Programm

Programmieren mit Python Turtle

Zum Einstieg verwenden wir das Python-Modul `turtle`. Damit können wir eine virtuelle Schildkröte über eine Zeichenfläche bewegen und dabei Linien und Figuren zeichnen.

Ein **Modul** ist eine Sammlung von bereits vorhandenen Funktionen und Befehlen, die in einem Programm verwendet werden können.

Grundgerüst

```
import turtle

t = turtle.Turtle()

# Schreibe deinen Code unter diese Zeile.
t.forward(60)
```

Import

```
import turtle
```

Mit dieser Zeile wird das Modul `turtle` in das Programm geladen. Erst danach können wir seine Funktionen verwenden.

Eine Turtle erzeugen

```
t = turtle.Turtle()
```

Damit wird eine Turtle erzeugt. Wir geben ihr den Namen `t`. Zu Beginn befindet sie sich in der Mitte der Zeichenfläche und schaut nach rechts.

Die Turtle bewegen

```
t.forward(50)
```

Dieser Befehl bedeutet:

> Bewege die Turtle 50 Einheiten vorwärts.

Beim Bewegen zeichnet die Turtle eine Linie.

Ein vollständiges erstes Programm sieht damit so aus:

```
import turtle

t = turtle.Turtle()
t.forward(50)
```

Wenn die Turtle auch als Schildkröte dargestellt werden soll, kann das Programm ergänzt werden:

```
import turtle

t = turtle.Turtle()
t.shape("turtle")
t.forward(50)
```

Kommentare

Kommentare sind Hinweise im Programmcode, die von Python nicht ausgeführt werden.

Ein Kommentar beginnt mit dem Zeichen `#` :

```
# Dies ist ein Kommentar.
```

Das Zeichen kann auch nach einem Befehl stehen:

```
t.forward(50)  # Die Turtle bewegt sich 50 Einheiten vorwärts.
```

Python ignoriert alles, was in dieser Zeile nach `#` steht.

Kommentare helfen dabei,

- Programme verständlicher zu machen,
- wichtige Stellen zu erklären,
- einen Programmablauf zu planen.

> **Merke:**
> Kommentare sind für Menschen gedacht, nicht für den Computer.

Programme lesen

Bevor ein Programm ausgeführt wird, sollte man versuchen, seinen Ablauf zu verstehen.

Beispiel 1

```
import turtle

t = turtle.Turtle()

t.forward(100)
t.left(90)
t.forward(50)
t.backward(100)
```

Überlege:

- Welche Linien zeichnet die Turtle?
- Wie lang sind die einzelnen Strecken?
- Wo befindet sich die Turtle am Schluss?
- In welche Richtung schaut sie?

Beispiel 2

```
import turtle

t = turtle.Turtle()

t.forward(100)
t.left(120)
t.forward(100)
t.left(120)
t.forward(100)
t.left(120)
```

Überlege auch hier zuerst ohne Ausführen des Programms:

- Welche Figur entsteht?
- Wo befindet sich die Turtle am Schluss?
- In welche Richtung schaut sie?

> **Tipp:**
> Zu Beginn befindet sich die Turtle in der Mitte der Zeichenfläche und schaut nach rechts.

Programme testen

Nachdem du ein Programm zuerst gedanklich untersucht hast, kannst du es in einer Entwicklungsumgebung ausführen.

Vergleiche das Ergebnis mit deiner Vorhersage:

- War deine Zeichnung korrekt?
- Falls nicht: Wo lag dein Überlegungsfehler?
- Welche Anweisung hattest du anders interpretiert?

Programme zu lesen und ihren Ablauf vorherzusagen ist eine wichtige Fähigkeit beim Programmieren.

Debugging

Beim Programmieren können unterschiedliche Fehler entstehen: ein Tippfehler, ein falscher Befehl oder eine falsche Überlegung.

Solche Fehler werden häufig als **Bugs** bezeichnet. Das systematische Suchen und Beheben von Fehlern nennt man **Debugging**.

Vorgehen beim Debugging

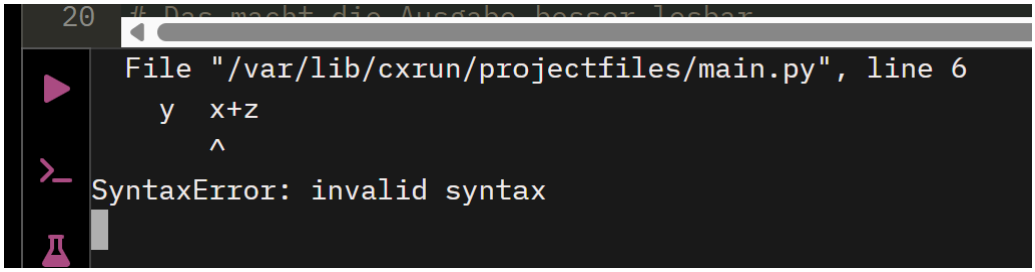
1. **Fehler bemerken**
   Das Programm startet nicht oder liefert ein unerwartetes Ergebnis.
2. **Fehler suchen**
   Analysiere den Programmcode Schritt für Schritt. Bei einer Fehlermeldung hilft häufig die angegebene Zeilennummer.
3. **Fehler beheben und testen**
   Ändere den Code und prüfe erneut, ob das Programm nun korrekt funktioniert.

Hilfsmittel beim Debugging

Fehlermeldungen lesen

Bei Syntaxfehlern zeigt Python eine Fehlermeldung an. Sie enthält häufig eine Zeilennummer und einen Hinweis auf die Stelle, an der der Fehler erkannt wurde.

> 👉 **Tipp:** Prüfe zuerst die angegebene Zeile und danach auch die unmittelbar davorliegende Zeile.



Beispiel einer Fehlermeldung, oft wird die korrekte Zeile des Fehlers angegeben

`print()` zum Testen

Mit `print()` können Zwischenwerte ausgegeben werden:

```
print("Variable x =", x)
```

Damit lässt sich prüfen, welchen Wert eine Variable gerade besitzt oder ob ein bestimmter Programmteil erreicht wurde.

Debugger

Viele Entwicklungsumgebungen besitzen einen **Debugger**. Damit kann ein Programm Schritt für Schritt ausgeführt werden. So lässt sich beobachten, welche Anweisungen ausgeführt werden und wie sich Werte verändern.

Syntaxfehler und semantische Fehler

Syntaxfehler

Ein **Syntaxfehler** entsteht, wenn die Schreibregeln der Programmiersprache verletzt werden.

Beispiel:

```
print("Hallo"
```

Hier fehlt eine schliessende Klammer.

Semantischer Fehler

Bei einem **semantischen Fehler** ist der Code zwar syntaktisch korrekt, das Programm macht aber nicht das, was beabsichtigt war.

Beispiel:

```
t.left(60)
```

Der Befehl ist korrekt geschrieben. Soll die Turtle jedoch für ein Quadrat um 90° drehen, ist die gewählte Zahl inhaltlich falsch.

> **Merke:**
> Ein Programm kann syntaktisch korrekt sein und trotzdem ein falsches Ergebnis liefern.

Aufgaben

Aufgabe 1 – Programme lesen

Untersuche die beiden Turtle-Programme oben, ohne sie auszuführen.

1. Zeichne jeweils die entstehende Figur.
2. Beschrifte die Seitenlängen.
3. Markiere die Endposition der Turtle.
4. Zeichne ein, in welche Richtung die Turtle am Schluss schaut.

Zusammenfassung

> **Merke**
>
> - Ein **Algorithmus** ist eine eindeutige Schritt-für-Schritt-Anleitung.
> - Ein **Computerprogramm** ist ein Algorithmus, der in einer Programmiersprache formuliert wurde.
> - Die **Syntax** beschreibt die Schreibregeln einer Programmiersprache.
> - Die **Semantik** beschreibt die Bedeutung eines Programms.
> - Ein **Modul** stellt zusätzliche Funktionen und Befehle bereit.
> - **Kommentare** helfen Menschen, Programme zu lesen und zu verstehen.
> - **Debugging** bedeutet, Fehler systematisch zu suchen und zu beheben.

Programmieraufgaben

Unter folgendem Link sind die Programmieraufgaben in CodeExpert zu finden:

Turtle_Basic

Turtle_Advanced